

Fast-Charging Station Simulator

This repository implements the stochastic models in Paul F. R. Saugier's 2017 thesis, *Design of a Fast-Charging Station for Electric Vehicles*, as a reproducible Python simulation and testing environment.

Implemented thesis models

- Non-homogeneous hourly Poisson motorway traffic (Table A1.1)
- Shifted-lognormal EV battery capacities and three vehicle classes
- Class-dependent inverse-lognormal initial state of charge
- Physics-based 100 km motorway energy model (section 2.3.4)
- The 20% reserve charging decision and charge to 80% SOC
- 200 kW constant-power (CP) charging
- 200-to-50 kW CC/CV charging, tapering after 60% SOC
- FIFO M/G/c queue with configurable chargers and plug/unplug dead time
- Station power, queue, utilization, session, and daily-energy traces
- Ideal, grid-limited energy-storage capacity sizing
- Erlang-C M/M/c results and the thesis's M/G/c waiting-time approximation

The defaults are the thesis's 30% EV-penetration case: ten 200 kW chargers, 90% charger efficiency, two minutes of dead time, and an 800 kW grid connection.

Quick start

Python 3.10+ and NumPy are required.

```
python -m pip install -e .
python -m fcsim --days 100 --profile cccv --penetration 0.30 --
output output/base-case
```

Reproduce the thesis's 100-day, constant peak-hour queue experiment:

```
python -m fcsim --peak-hours 2400 --profile cccv --output
output/peak-cccv
```

For a 40% penetration design with twelve chargers:

```
python -m fcsim --days 100 --penetration 0.40 --chargers 12 --
output output/40pct-12
```

Run the validation suite:

```
python -m unittest discover -s tests -v
```

The CLI prints a JSON summary and writes:

- `summary.json` and `days.csv` for Monte Carlo runs
- `summary.json`, `timeseries.csv`, and `sessions.csv` for a one-day run

All inputs can also be changed through `SimulationConfig` in Python, including the 24 hourly traffic rates, charger efficiency and power, SOC thresholds, dead time, grid limit, and sample interval.

```
from fcsim import ChargingProfile, SimulationConfig,
run_monte_carlo

config = SimulationConfig(
    penetration=0.40,
    chargers=12,
    profile=ChargingProfile.CCCV,
    grid_limit_kw=1000,
)
result = run_monte_carlo(config, days=100, seed=2026)
```

```
print(result.summary())
```

Model notes

The implementation follows the thesis's stated equations and parameters. Two details are made explicit:

1. The inverse-lognormal SOC distributions have unbounded tails, so extremely rare nonphysical values are clipped to `[0, 1]`.
2. The CC/CV figure specifies a power taper linear in time. The simulator integrates and analytically inverts that profile, including vehicles that arrive above 60% SOC.

By default, arrivals use the published station-rate column of Table A1.1, which reproduces the thesis's stated 17.4% stopping rate. Set `use_published_station_rates=False` to generate every motorway EV and derive the stop decision entirely from the battery/SOC/consumption model. With the rounded parameters printed in the thesis, that bottom-up mode produces a somewhat lower stopping fraction; keeping both modes makes this discrepancy visible rather than hiding it in a calibration constant.

Storage is technology-neutral, matching the thesis: it assumes no losses or power limits and reports the minimum usable capacity required when the store begins full and grid input is capped. This is a screening model, not a battery degradation or converter design model.

Thesis-to-code mapping

Source: Paul F. R. Saugier, *Design of a Fast-Charging Station for Electric Vehicles*, University of Cambridge, 2017.

Thesis material	Implementation
2.1 Poisson process and M/G/c queue	<code>analytics.py</code> , <code>queueing.py</code>
2.2 hourly motorway and station flow; Appendix A1.1	<code>config.py</code> , <code>simulation.py</code>
2.3.2 shifted-lognormal capacity and EV classes	<code>vehicles.py</code>
2.3.3 inverse-lognormal initial SOC	<code>vehicles.py</code>
2.3.4 vehicle force, mass, auxiliary load, trip energy	<code>motorway_energy_kwh</code> in <code>vehicles.py</code>
2.3.5 20% reserve decision and 80% target	<code>sample_vehicle</code> in <code>vehicles.py</code>
2.3.6 CP and CC/CV profiles	<code>charging.py</code>
2.4 ten-server FIFO and two-minute dead time	<code>dispatch_fifo</code> in <code>queueing.py</code>
2.5 peak and daily simulations	<code>simulate_peak</code> , <code>simulate_day</code> , <code>run_monte_carlo</code>
2.6 ideal storage behind an 800 kW grid tie	<code>storage.py</code>
Appendix 2 M/M/c / Erlang C	<code>erlang_c</code> in <code>analytics.py</code>

Reproduction checks

The following checks used deterministic seeds and the published station-arrival rates. Monte Carlo values vary slightly by seed and run length.

Quantity	Thesis	Simulator check
Fraction of motorway BEVs stopping	17.4%	17.2-17.4%
Mean CP charge	7.0 min	7.03 min

Mean CC/CV charge	9.1 min	9.17 min
30% daily energy	12 MWh	about 12 MWh
40% daily energy	16 MWh	16.2 MWh
40% CC/CV mean usable storage	3 MWh	3.34 MWh
40% CC/CV 95th-percentile usable storage	4 MWh	4.09 MWh

Explicit interpretations

- The blue curve in Figure 2.8 is treated as a charging-power taper linear in time from 200 kW at 60% SOC to 50 kW at 80% SOC. Energy is integrated analytically.
- Charger power is station-side input; battery-side power is reduced by the stated 90% efficiency.
- A charger becomes available only after charging plus dead time. Power is zero during dead time.
- Hourly arrivals are conditionally uniform within each hour, which is equivalent to a homogeneous Poisson process over that hour.
- The ideal storage model starts full, accepts surplus grid energy until full, spills excess, and has no loss or power limit, matching section 2.6.
- The published station rates are the default reproduction path. The bottom-up stopping decision remains available with `use_published_station_rates=False`.

Deliberately out of scope

The thesis does not specify a finite queue, customer abandonment, battery aging, storage efficiency, storage power limits, converter losses beyond charger efficiency, renewable generation, tariffs, or economic optimization. Those are therefore extension points rather than silently invented behavior.